

Работа с външни файлове

I. Теоретична постановка

1. Файл

Файлът е съвкупност от данни (поредица от байтове), записани върху постоянната памет на компютъра (например твърдия диск).

Използваме файлове, когато е нужно да съхраним генерираните от програмата данни и след приключване на работата на програмата или когато трябва да прочетем входните данни за работата на програмата от външен спрямо приложението източник.

2. Операции при работа с файл

Операциите при работа с файл включват следните действия: отваряне на файла, четене или запис на данни и затваряне на файла.

- Отварянето може да стане с вградената функция `open()`:

```
open(file, mode="r")
```

Основните параметри на функцията са път и име на отваряния файл и режим на достъп.

Ако файлът е в работната папка на приложението, то е нужно да се посочи само неговото име заедно с разширението.

```
f=open("test.txt", "r")          # Отваря файл с името test.txt от работната папка
```

```
f=open("C:\\Folder\\test.txt","r")  # Отваря файл с името test.txt, който е на устройство C: в  
                                     # папка Folder (Разделителят '\\' в пътя до файла на Windows  
                                     # е специален символ и трябва да се екранира)
```

Режимите за достъп са няколко вида:

"r" – (по подразбиране) Четене. Отваря файла за четене. Дава грешка при несъществуващ файл.

"w" – Запис. Отваря файла за запис. Създава файла, ако той не съществува.

"a" – Добавяне. Отваря файла за добавяне в края. Създава файла, ако той не съществува.

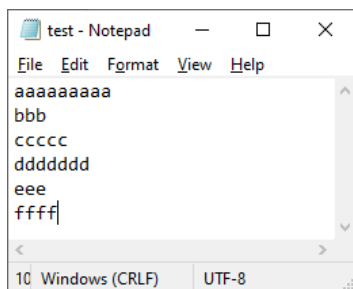
"x" – Създаване. Създава файла. Дава грешка ако файла вече съществува.

В добавка към тези символи в режима за достъп могат да се добавят и символите "t" (текстов файл) и "b" (двоични файлове). Напр. "rt", "wt", "rb", "wb" и др.

Освен основните параметри на функция `open()` в списъка може да се добави и параметър `encoding`. Чрез него се задава кодировката на информацията във файла. Най-често се използва `encoding="utf-8"`. Това се задава само при работа с текстови файлове. Ако искате да записвате или четете текстове на кирилица във файл трябва да зададете точно тази кодировка!

- Четенето може да стане по няколко начина.

Функция `open()` връща обект файл и за него могат да се извикат различни методи за четене.
`read(<Количество байтове>)` – прочита зададеното количество байтове от файла. Ако не бъде посочено количество, се четат всички байтове до края на файла.
`readline(<количество>)` – прочита един ред от файла. При подадено количество се четат определен брой байтове от реда. В противен случай се прочита целия ред. Ако файлът е текстов, методът връща низ. При двоичен файл – връща последователност от байтове.
`readlines()` – Чете съдържанието на файла и го записва в списък.



```
f = open("test.txt", "r")
print(f.read())           # Чете целия файл
```

```
f = open("test.txt", "r")
print(f.read(5))          # Чете 5 символа
```

```
f = open("test.txt", "r")
print(f.readline(), end="") # Чете първи ред
print(f.readline(), end="") # Чете втори ред
```

```
f = open("test.txt", "r")
print(f.readlines())       # Връща списък със записаните във файла низове
```

Обхождане на данните може да се направи и като просто итерираме елементите в самия обект файл. Това е най-компактния и удобен начин!

```
f = open("test.txt", "r")
for x in f:
    print(x, end="")
```

- Записът на файла също може да стане по няколко начина

`write(<Данни>)` – записва данни (низ или последователност от байтове) във файл като изтрива старото му съдържание ако е в режим “w” или добавя данни към съдържането ако е в режим “a”.

`writelines(<последователност>)` – записва последователност във файл. Ако всички елементи на последователността са низове, файлът може да бъде отворен в текстов режим. В противен случай файлът трябва да бъде отворен в двоичен режим.

```
f=open("test.txt","w")
f.write("New strings!")    # Запис на низ
f.close()
```

```
f=open("test.txt","a")
f.write("\nNew strings!")          # Добавяне на низ
f.close()
```

```
f=open("test.txt","w")
f.writelines(["String1\n","String2\n", "String3"])      # Запис на списък
f.close()
```

- Затваряне на файла
Прави се с метод close().

```
f = open("test.txt", "r")
print(f.readline())
f.close()
```

Затварянето на файла прекъсва връзката между програмата и физическия файл. То не трябва да се пропуска, защото чрез него се записват всички направени промени с данните в него. Ако използваме оператор `with` при отварянето на файла, затварянето на файла става автоматично и не е нужно да се вика метод `close()`. Това е препоръчителният начин на работа!

```
with open("test.txt","r") as f:
    print(f.read())
```

- Изтриване на файл
Файл може да бъде изтрит с помощта на функция `remove()` от модул `os`:
`import os`
`os.remove("test.txt")`

Преди да се изтрие файл, трябва да се провери дали той съществува. В противен случай ще получим съобщение за грешка.

```
import os
if os.path.exists("test.txt"):
    os.remove("test.txt")
else:
    print("The file does not exist")
```

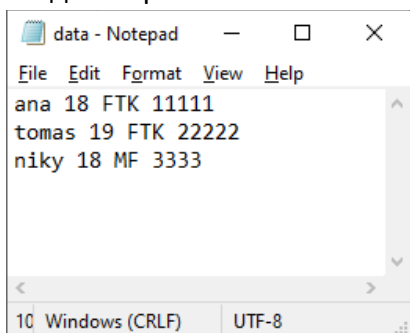
Пример 1: Да се напише програма, която чете 5 низа от клавиатурата и ги записва в текстов файл. Програмата да отвори файла и да разпечата неговото съдържание.

```
with open("test.txt","w",encoding="utf-8") as f:      # Отваряне на файла за запис
    for i in range(0,5):
        str=input(f"Низ {i+1}: ")
        f.write(f"{str}\n")
with open("test.txt","r",encoding="utf-8") as f:      # Отваряне на файла за четене
```

```
print(f.read())
```

Пример 2: Да се напише програма на Python, която създава клас студент с атрибути име и възраст (цяло число) и име на университет - ТУ. В класа да се включат метод за извеждане на данните за студента. Да се създаде клас наследник факултет с атрибути име на факултет и факултетен номер. Да се прочете информацията за студентите от текстов файл. Да се изведат основните данни (име и възраст) за всички студенти на ТУ. Да се изведат всички записани данни за студентите само от избран факултет.

Забележка: Преди да стартирате програмата отворете текстов редактор (напр. Notepad) и създайте файл data.txt със следното съдържание:



```
class Student:                                # Клас студент
    """Клас за студенти в ТУ"""
    university="Технически университет - София"
    def __init__(self,name,age):
        self.name=name
        self.age=age
    def show_st(self):
        print(f"{self.name} {self.age}")
class Faculty(Student):                        # Клас факултет
    """Клас за студенти от факултет"""
    def __init__(self,name,age,fk_name,number):
        super().__init__(name,age)
        self.fk_name=fk_name
        self.number=number
    def show_fk(self):
        print(f"{self.name} {self.fk_name} {self.number}",end="")
    def get_fk_name(self):
        return self.fk_name

def add_student(studenti):                    # Функция за четене на данни за студенти от файл
    """Четене на данни за студенти от файл"""
    with open("data.txt","r",encoding="utf-8") as f:        # Отваряне на файла
        for s in f:                                          # Обхождане на файла ред по ред
            ls=s.split(" ")                                # Създаване на списък от думите, разделени празен интервал
            F=Faculty(ls[0],ls[1],ls[2],ls[3])              # Създаване на обект от клас факултет
            studenti.append(F)
```

```
    return studenti
def show_student(st_list):    # Функция за извеждане на основни данни всички студенти
    """Извеждане на списък със студенти на ТУ"""
    if st_list!=[]:
        print(Student.university)
        for i in range(0,len(st_list)):
            st_list[i].show_st()
    else:
        print("Списъкът е празен!")
def show_fakulty(st_list,fk):    # Функция за извеждане на студенти от избран факултет
    """Извеждане на списък със студенти във факултет"""
    flag=0
    for i in range(0,len(st_list)):
        if st_list!=[]:
            fname=st_list[i].get_fk_name()
            if fname==fk:
                st_list[i].show_fk()
                flag=1
        else:
            print("Списъкът е празен!")
    if flag==0:
        print("Няма записани студенти от избрания факултет!")

lst=[]    # Създаване на празен списък
print(add_student.__doc__)
lst=add_student(lst)
print(show_student.__doc__)
show_student(lst)
key=input("Въведете име на факултет: ")
print(show_fakulty.__doc__)
show_fakulty(lst,key)
```

Тествайте програмата и я сравнете с Пример 2 от Упражнение 11. По какво си приличат и по какво се различават двете програми? При коя програма данните за студентите остават записани и след края на работата на програмата и могат да се ползват отново от нея или от друго приложение?

Пример 3: Да се напише програма на Python, която чете файла data.txt от Пример 2 и намира средната възраст на студентите от ТУ.

```
suma=0
index=0
with open("data.txt","r",encoding="utf-8") as f:    # Отваряне на файла за четене
    for s in f:    # Обхождане на файла ред по ред
        index+=1
```

```
ls=s.split(" ")          # Запис на данните в списък
print(f"{ls[0]} {ls[1]} {ls[2]} {ls[3]}",end="")
#num=int(ls[1])          # Преобразуване на данните за възраст до цяло число
suma+=int(ls[1])

try:
    av=suma/index          # Изчисляване на средната възраст на студентите
    r="%.2f"%(av)
    print(f"\nЗаписани са {index} студенти със av = {r}")
except ZeroDivisionError:
    print(f"Деление на нула! Записани са {index} студенти!")
```

Изтрийте всички записи от файла data.txt и стартирайте отново програмата. Какво се промени в изпълнението на програмата?

II. Задачи за самостоятелна работа

1. Направете промени в кода на Пример 1, така че четенето на файла да става

А) ред по ред (Използвайте параметъра end="" на функция print(), за да не остава по един празен ред след всеки низ)

Б) в списък

2. Напишете програма, която да създава текстовия файл data.txt, използван в Пример 2. Спазете съдържанието на данните, показано в примера.

Помислете как да изведете данните на студенти от избран факултет, които са и от избран курс.

3. Направете промени в кода на Пример 3, така че да се определя средната възраст на студентите от избран факултет.